The Reality Code Engine: One Form, Many Fits, and the Low-Entropy Path to Autonomous Coding

David Charlot, Open Interface Engineering

Research paper · OpenIE research.openie.dev
https://research.openie.dev/papers/rce/ · PDF https://research.openie.dev/pdfs/rce.pdf

Measurement. Research paper, not a journal final. Handler figures are est_j. Track B figures are reported_j from a RAPL package counter less a stated idle baseline. measured_j is reserved for a wall meter and is empty. No FPGA board result in this paper. No fabricated citations.

There are two paths to autonomous computer automation. One spends high energy on high entropy. The other converges on a low-entropy floor. A program is a database functional state machine: one Form, many fits. The Reality Code Engine (RCE) retrieves, adapts and proves fits instead of generating them. Correctness is the gate. Joules are the ranking. On held-out test splits with the engine frozen and 0 model calls, it repairs 520,541 of 520,821 one-token faults in nine real repositories (0.9995, 95% interval 0.9994 to 0.9995), 23,015 of 23,015 in four repositories never used for tuning, 38.6% of 24,072 single-statement Java bugs from ManySStuBs4J and 17.4% of 157,089 Python bugs from TSSB-3M.

Version 1.1, 11 October 2026. Data and code: research.openie.dev/data/.

One Form. Many fits. The gate is correctness, and the ranking is joules.

The economic reading of this paper is in its companion, The Promised Trillion Dollar AI-Coding Market. This paper holds the engineering and the theory only.

---

1. Thesis

There are two paths to autonomous computer automation. They are told apart by two quantities: the energy spent per correct result, and the entropy the system must resolve at run time.

The first path generates. A large language model (LLM), a statistical text model, writes each change from the prompt. Every task pays for generation again, and every output is a fresh sample from a high-entropy distribution.

The second path converges. People have already coded most of what will ever be coded. A program is a database functional state machine: one Form, many fits. Human intent is high entropy. The machine floor is low entropy. Code converges by design, so most of the work is to retrieve a fit that exists, adapt it, and prove it. The Reality Code Engine (RCE) is built on the second path.

Both paths face the same gate. The gate is correctness: a task's completeness predicate $C(z)$, here the task's tests, a proof or an agreement trace. Among runs that pass the gate, systems are ranked by joules to completion, J^*. Correctness is a constraint, not a score.

Mixture of Limits is the philosophy: there is a minimal amount of anything to accomplish a task, and any more is waste (Metered Commit Gates, Part II). RCE is a Mixture of Limits instance. Its gears run cheapest first, and the first gear that passes the gate closes the task.

flowG is a stand-alone language project. RCE is its academic first-pass demonstration. The two stay separate projects.

Energy figures are labeled by source. `est_j` is an analytical estimate. `reported_j` is an operating-system or on-chip counter. `measured_j` is reserved for a wall meter and is empty in every result here.

2. Two paths to autonomy

One sentence in. One floor out.

How human intent becomes something a computer can run, and why it loses meaning on the way.

1. HUMAN INTENT IS HIGH ENTROPY

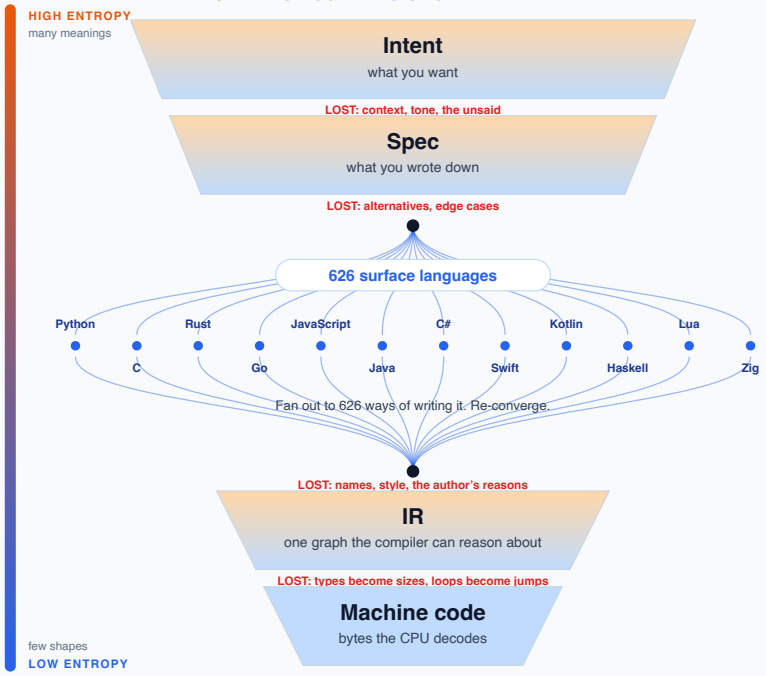
“Make the checkout faster.”

- Cut page load time?
- Fewer steps for the buyer?
- Faster card approval?
- Cache the cart?
- Faster for whom: phone or desktop?

Many meanings live in one sentence.
Context is implied, never written.
The listener fills the gap.
A computer cannot.

Shannon measured English at 0.6 to 1.3 bits per letter.
The letters are predictable. The meaning is not.

2. THE FUNNEL OF CONVERSIONS



3. It all gets to assembly.

`mov` `add` `cmp` `jmp` `load` `store` `call` `ret`

4. WHY ALL CODE STARTS TO LOOK THE SAME

The destination has room for only a few shapes.

Every program ends as loads, stores, math and jumps.
So code bends toward those shapes.

Convergence is by design. Not by laziness.

70% **of code on GitHub is clones.**
428M files, 85M unique.

Déjà Vu, Lopes et al., OOPSLA 2017

5. WHERE THE LOSS HAPPENS

Same line. Different answer.

Operators agree on easy inputs.
They split at the edges.

expression	Python	C
`-3 % 2`	`1`	`-1`
`-7 / 2`	`-3.5`	`-3`
`MAX + 1`	`grows`	`undefined`

Every boundary is a guess about what you meant.

Most guesses are right. Bugs live in the rest.

6. THE FIX: RCE

Stop guessing at every boundary.

1. **One canonical form**
   Write the meaning once.
   One structure holds the intent.
2. **Every surface is a projection**
   Python, Rust, WASM, XState.
   Each prints byte-identical traces.
3. **Cheapest correct form wins**
   Equivalent forms, ranked by joules.
   Pick the one that costs least.

Sources: Shannon, “Prediction and Entropy of Printed English,” Bell System Technical Journal, 1951. Lopes et al., “DéjàVu: A Map of Code Duplicates on GitHub,” OOPSLA 2017.
626 languages: David Charlot’s lowering-path survey. Python and C results are language spec behavior. C signed overflow is undefined behavior.

Figure 1. Intent entropy and the machine floor. A PNG copy is provided. Sources: Shannon (1951); Lopes et al. (2017); the 626-language count is David Charlot's lowering-path survey, which is unpublished; its list and method are to be published with this paper's data, and until then the count is the author's. The Python and C results are language specification behavior; C signed overflow is undefined behavior.

Intent is high entropy. "Make the checkout faster" carries many meanings: shorter page load, fewer steps, faster card approval, a cached cart, or faster on one device class. Context is implied and never written. Shannon measured printed English at about 0.6 to 1.3 bits per letter [1]. The letters are predictable. The meaning is not.

Every conversion loses information. Intent becomes a specification, and context is lost. The specification becomes one of hundreds of surface languages, and alternatives and edge cases are lost. The source becomes an intermediate representation (IR), and names, style and the author's reasons are lost. The IR becomes machine code, and types become sizes and loops become jumps.

The floor is low entropy. Every program ends as loads, stores, arithmetic and jumps. The destination has room for only a few shapes, so code bends toward them. Convergence is by design.

Boundaries are where meaning diverges. The same line gives different answers at the edges. In Python, `-3 % 2` is 1 and `-7 / 2` is -3.5. In C, `-3 % 2` is -1, `-7 / 2` is -3, and signed overflow is undefined. A system that regenerates code at every boundary guesses at every boundary.

The two paths, stated as quantities.

Path	What runs per task	Entropy resolved at run time	Energy per correct result
Generate	A model samples a new change	The full distribution of the prompt	Paid again on every task
Converge (RCE)	Lookup, rule, solver; a model only last	Only the residual that no stored fit covers	Paid once per new case; then the handler floor

RCE writes the meaning once, in one canonical Form. Every surface language is a projection of that Form, checked to match it. Among equivalent forms, the cheapest correct one runs.

2.1 Programming is coding theory

The thesis of this section is David Charlot's: all of programming is coding theory. A programmer converts intent across two entropic regimes. Intent is high entropy, a distribution over meanings. Code is low entropy, one deterministic transition per state. Every step of software work, from a requirement to a register, is a step of that conversion. The literature has said so in pieces for seventy years, in many languages. This section assembles the pieces into one argument and places each RCE gear on it.

The measure. Shannon defined the entropy of a source and the capacity of a channel [40], and measured printed English at about 0.6 to 1.3 bits per letter [1]. Kolmogorov, writing in Russian in *Problemy Peredachi Informatsii* in 1965, gave three approaches to the quantity of information and made the algorithmic one primary: the information in an object is the length of the shortest program that produces it [41]. Solomonoff had reached the same object as a theory of inductive inference [42], and Chaitin showed that program size obeys identities formally identical to Shannon's [43]. Rissanen turned the idea into a rule for choosing models, the minimum description length [44]. Together they say that a program is a code word: a description of a behavior, and the shortest correct one is the floor. RCE's extraction rule, keep the least-joule fit among equivalent fits (Proposition 3), is that rule with joules in place of bits.

Code sits in the low-entropy regime, and people measured it. Hindle, Barr, Su, Gabel and Devanbu found that code is "very repetitive, and in fact even more so than natural languages," at lower cross-entropy under the same n-gram models [35]. Gabel and Su found that most lines of a project already exist elsewhere [34], and Lopes et al. found that most files on GitHub are clones [33]. Tu, Su and Devanbu found that code is also local: what was written nearby predicts what comes next [46]. Ray and colleagues found that buggy lines are less natural, with higher entropy, and that the entropy falls when the bug is fixed [45]. In 2025 Yang, Kolak, Hellendoorn and Martins used the entropy change a patch induces to rank repair candidates and to separate correct patches from overfitting ones [51]. Allamanis, Barr, Devanbu and Sutton surveyed the field under the name naturalness [47]. The result is not only an English-language one. Prem Devanbu stated it as a keynote at the India Software Engineering Conference in 2013 [50]. Sridhara, Sinha and Mani at IBM Research India measured the natural-language artifacts around code at the same venue and found that commit messages are the most regular and Stack Overflow prose behaves like ordinary English [49]. In China, the *Journal of Software* (软件学报) published a review of code naturalness research in 2022, which states the cross-entropy measure and the finding that defective code has higher entropy [48]. In Japan, Doi, Higo, Arima, Shimonaka and Kusumoto at Osaka University used n-gram naturalness, first in a Japanese-language IEICE technical report and then at ICPC 2018, to tell machine-generated source files from human-written ones [73, 74]. Generated code sits even lower on the entropy scale than human code, because a generator is a fixed conversion. In Brazil, Arrebola's doctoral thesis built code suggestion on n-gram and topic models segmented by project and maintainer, and evaluated it by cross-entropy, the bits needed to encode the next token [75]. The two regimes are visible in one repository: the prose around the code is high entropy, and the code is low.

The measure matters. Febres, Jaffé and Gershenson, working from Venezuela and Mexico, compared English, Spanish and program text by vocabulary diversity per message length and found the program text more diverse [76]. Rahman, Palani and Rigby found that much of the n-gram predictability of code comes from syntax tokens, and that code is still repetitive, less than first reported, once those are removed, with API usage the most repetitive part [77]. Both results sharpen the claim rather than reverse it. Diversity of symbols is not predictability, and the low-entropy regime is a property of the conditional distribution of the next step, which is what a deterministic transition fixes. RCE's G0 keys on shapes, not on surface tokens, for that reason.

Intent sits in the high-entropy regime. Zave and Jackson showed that a requirement is a statement about the environment, a specification is a statement at the interface, and the gap between them is filled only by domain knowledge that is often unwritten [64]. Brooks split the difficulty of software into essence and accident: the essence is the conceptual construct, and most of it is specification and design, not representation [61]. Naur argued that a program is the visible part of a theory held by its programmers, and that the theory is not fully recoverable from the text [62]. Read through Shannon, all three say the same thing. Intent carries more uncertainty than any one program, and writing the program is the act that removes it. What remains in a programmer's head after the code is written is the part of the intent the code did not encode.

The conversion is a refinement, and refinement is a narrowing. Hoare gave programs a logic in which a program is correct relative to a pre- and postcondition [54]. Wirth taught stepwise refinement: decompose a task into instructions by successive decisions, each one fixing a choice the specification left open [53]. Dijkstra's guarded commands made the open choices explicit as nondeterminacy, and derived programs from specifications by removing it [52]. In Munich, Bauer and Wössner's CIP project built program development as a sequence of correctness-preserving transformations from specification to program [58]. In France, Abrial's B method and then Event-B carried refinement to industrial scale, with proof obligations at each step [55, 56]. In Finland, Back and von Wright gave refinement its lattice-theoretic calculus [57]. In each, a specification denotes a set of acceptable behaviors and each refinement step shrinks the set, until one deterministic behavior remains. That is entropy reduction with a proof at every step. RCE's Form is the last element of that chain: a total, deterministic transition δ over database states, with every choice resolved (§3.2).

Synthesis is search that narrows a hypothesis space. Gulwani's FlashFill learns a string program from a few input-output examples by representing the set of all consistent programs and intersecting it with each new example [59]. Gulwani, Polozov and Singh frame program synthesis in general as search through a space of programs constrained by intent [60], and Solar-Lezama's sketching fills holes in a partial program with a solver [27]. Each example or constraint is a measurement that removes hypotheses. Shannon's measure counts how many such measurements are needed. RCE's G2 is that gear.

What the conversion produces has one shape. Gurevich proved that every sequential algorithm is captured by an Abstract State Machine at its own level of abstraction [2]. Codd made the state a relation [5], and event sourcing makes the state the fold of an append-only log [14]. Under Curry and Howard, a proof is a program and a proposition is a type [18, 72], so a verified fit is a proof of its specification. These are the reasons the low-entropy regime has few shapes: the destination is a state machine over a database, whatever the surface language.

Low entropy decays without a gate. Lehman observed over decades of large systems that a program in use must keep changing, and that its complexity grows unless work is done to reduce it [63]. Torres, Baltes, Treude and Wagner measured structural and textual entropy over the commit histories of 25 projects and found the two rise together [78]. Entropy flows back into code from the environment. A system that keeps code in the low-entropy regime needs a gate on every change, and RCE's gate is correctness (§4.3).

Compilers already do source coding. Franz and Kistler shipped programs as compressed abstract syntax trees and generated native code on load, which is source coding of a program over its own grammar [67]. Fraser inferred statistical models of code from a corpus to compress it [68]. Futamura showed in 1971, in the Japanese journal *Systems, Computers, Controls*, that specializing an interpreter to a program yields a compiled program, and that specializing the specializer yields a compiler [70]. Ershov in Novosibirsk developed the same idea as mixed computation [71]. The Cousots' abstract interpretation computes a sound, lower-information description of every behavior of a program [69]. The analogy to channel coding is this paper's reading, not a cited theorem: types, tests and proofs add checked redundancy so that a wrong program is detected before it runs, as a parity check detects a corrupted word.

Erasure is where the joules go. Landauer showed that a logically irreversible step, the erasure of one bit, dissipates at least $k_B T \ln 2$ [65]. Bennett showed that computation can in principle be made logically reversible, so the erasure, not the computation, carries the bound [66]. Converting high-entropy intent to one low-entropy program discards every alternative the intent allowed. That discard is an erasure of information about the alternatives, and it has a thermodynamic floor. The agency paper prices each confirmed bit against that floor. RCE pays the discard once per new case and stores the result, so a repeat case erases nothing new.

2.2 How RCE converts, and where each gear sits

Read as coding theory, a task arrives as a message whose remaining uncertainty is what no stored fit already resolves. Each gear removes uncertainty at a different price, and the gears are ordered by that price (§4.2).

Gear	What it does to the uncertainty	Lineage in the argument above
G0 Lookup	Recognizes that the intent was already encoded; resolves it by retrieval, at the handler floor	Repetitiveness and clones [33, 34, 35]; locality [46]
G1 Rules	Applies a known guarded rewrite; resolves a recurring residual with no search	Correctness-preserving transformation [52, 58]; refinement with proof obligations [55, 56]
G2 Synthesis	Searches a hypothesis space and narrows it by examples, constraints and a solver	Program synthesis as search [27, 59, 60]
G3 to G5 Models	Samples from a learned distribution over code when no lower gear closes the gap; a proposer only	Cross-entropy models of code [35, 47]
Gate	Accepts a candidate only with tests, coverage and a proof certificate	Hoare logic [54]; Curry-Howard [18, 72]
Promotion	Stores the accepted fit, so the same residual costs G0 next time	Minimum description length [44]; Landauer erasure paid once [65]

The two paths of the table in Section 2 differ in where the conversion is paid. The generation path samples the full distribution of the prompt on every task, so it pays the high-entropy side again each time. The convergence path pays only for the residual, and only once. The economic reading of that difference is in The Promised Trillion Dollar AI-Coding Market, Section 4. The stop rule behind the gear order is Mixture of Limits, and the per-task stop is Task Satiation.

2.3 Per-rung ladder results

A coding task is measured here by the **intent bits** it needs: the bits that must be supplied from outside what the engine already holds (the program around the edit, the libraries, the tests, the index of known fixes). Writing H_{task} for the bits that pin the target artifact down and H_{corpus} for the bits the corpus already fixes, the supplied bits are $I = H_{task} - H_{corpus}$. A gear that ranks the right edit at rank r among its candidates has supplied $\log_2 r$ bits of search, and the gate supplies the rest. The ladder orders tasks by edit size, an objective proxy for I: one token (L0), one statement (L1), one hunk (L2), several hunks in one file (L3), several files (L4), a function from a specification or tests (L5), an issue-level change (L6) and a program from a specification (L7). The rungs, corpora, splits and hypotheses were preregistered before the full runs; the files are in the public repository.

Rung	Held-out test corpus	N	Solved	Rate	Wilson 95% CI	Median search bits	Model calls
L0 one token	PIT-style mutants [109] of 9 real repositories at pinned commits (Java: commons-math, commons-collections, jsoup, joda-time, gson; Python: click, attrs, pydantic, sympy)	520,821	520,541	0.9995	0.9994 to 0.9995	2.3	0
L0 one token, repositories never tuned on	commons-text, commons-io, more-itertools, httpx	23,015	23,015	1.0000	0.9998 to 1.0000	2.0	0
L1 one statement, Java	ManySStuBs4J 2.0 single-hunk fixes [107]	24,072	9,288	0.386	0.380 to 0.392	3.5	0
L1 one statement, Python	TSSB-3M v1.0.0, shards 14 and 15 [108]	157,089	27,266	0.174	0.172 to 0.175	4.8	0

L2 to L4 are measured on the Defects4J held-out set of 288 bugs (Section 8.1). L5 to L7 follow the protocols of Section 8.2. Section 6 gives the method, the gate and the energy of each row.

The rates read as intent bits. At L0 the missing bits are almost all on the line, and a prior-ordered single-token neighbourhood supplies them in a median of 2.3 bits. At L1 the engine sees only the 7-line hunk. It closes the categories whose bits are inside the hunk (Java: changed operator 91%, swapped boolean literal 84%, swapped arguments 82%; Python: changed boolean literal 98.8%,

swapped arguments 99.4%) and leaves open those whose bits are outside it: method names (22%) and identifiers (37%) need the file and its types, and new conjuncts (3% to 7%) and whole-statement rewrites (Python, 1.2% of a category that is 34% of the corpus) are new intent. New intent is what a supplier (a specification, a test, a person or a local model) provides once, after which the fit is in the index.

---

3. The Form

3.1 The strongest formal statement of "same form"

The scope of the theorem is stated first. Gurevich's sequential postulates cover sequential algorithms; parallel and interactive algorithms need the further postulates of Blass and Gurevich [3]. Blass, Dershowitz and Gurevich argue that no single equivalence relation on algorithms fits every purpose [79], and Moschovakis and Vardi set out the debate over what an algorithm is [80, 81]. A web handler that does input and output sits outside the sequential theorem. The Form handles interaction the way the interactive line does: each input is an event in E, and the state machine steps on it. So "one Form" is a modelling choice grounded in the sequential theorem and extended by convention, with production precedents in the Elm architecture [83], durable workflow event histories [84] and stream semantics such as Flo [82]. The propositions of Section 3.4 are stated for that choice.

Gurevich's Abstract State Machine (ASM) theorem is the strongest formal statement that every algorithm has one shape [2]. An abstract state machine is a machine whose states are first-order structures and whose step is a finite set of guarded updates. Gurevich states three postulates. Sequential Time: an algorithm has states, initial states and a one-step map τ. Abstract State: states are first-order structures over one fixed vocabulary, closed under isomorphism. Bounded Exploration: a finite set of terms decides each step. His Main Theorem (Theorem 6.13) reads: "For every sequential algorithm A, there exists an equivalent sequential abstract state machine B." Blass and Gurevich extend it to parallel algorithms [3]. Dershowitz and Gurevich add a fourth axiom, that initial operations are computable, and from it derive Church's thesis [4].

3.2 Definitions

Definition 1 (the Form). A program is a tuple

$$P = (S, E, \delta, D, \lambda)$$

- S, the state space: structures over a fixed vocabulary Σ. Each state is a database instance in Codd's sense [5].
- E, the event alphabet, including the logical clock tick.
- $\delta : S \times E \to S$, the transition: total, deterministic and pure, written as a finite set of guarded updates "if φ then $f(\bar{t}) := t_0$", each step reading a bounded set of locations.
- $D = (s_0, L)$, the durable store: an initial state and an append-only event log $L = e_1, \ldots, e_n$.

- $\lambda : S \times E \to O$, the output map. A Mealy machine reads both arguments [6]; a Moore machine reads the state only [7].

Definition 2 (Form and fit). The *form* is the tuple shape with four invariants: δ is deterministic and pure; exploration is bounded; the vocabulary is fixed; state is the fold of the log,

$$\text{state}(D) = \text{foldl}(\delta, s_0, L).$$

The *fit* is the data that fills the shape: the vocabulary Σ, the alphabets E and O, the rule program Π that defines δ, the output rules that define λ, and the initial data s_0. In Gurevich's terms the fit is the vocabulary, the ASM program and the initial structures. In Codd's terms Σ is the schema and s_0 is the instance. In product-line terms a fit is a feature selection [8]. In syntax-guided synthesis (SyGuS) terms a fit is a term from a grammar that meets a specification [9].

Definition 3 (fit equivalence). Two fits are equivalent when, for every log L, their output sequences under λ are equal.

Definition 4 (fit language). A grammar whose terms denote fits. Programming languages, SCXML documents, XState configurations and `.rce` specifications are fit languages. A fit language is complete when every ASM program over Σ has a term in it.

Definition 5 (control projection). Split $\Sigma = \Sigma_c \cup \Sigma_d$ into finite control symbols and data relations. The statechart of P is the projection of δ onto Σ_c: states are values of Σ_c, transitions carry an event and a guard over Σ_d, and actions are updates to Σ_d. Harel's statecharts give the hierarchy and the orthogonal regions [10].

Definition 6 (fit e-graph). For a fit F and a set R of sound rewrites, the fit e-graph $EG(F, R)$ is the saturated e-graph of F's rule terms under R [11, 12]. Each e-class holds fits equivalent by Definition 3.

Definition 7 (joule cost). $\text{cost}_J(t)$ is the estimated energy (`est_j`) to run term t once on a named substrate, weighted by the expected number of runs.

3.3 Why "database functional state machine"

A finite relational structure is a database instance, so a Gurevich state is a database and each step is a bounded transaction. *Database*: S is a relational structure. *Functional*: δ and λ are pure. *State machine*: runs are folds of δ over events. Moseley and Marks draw the same line in architecture [13]. Event sourcing draws it in application state: "Capture all changes to an application state as a sequence of events" [14]. Dedalus draws it in logic, where mutable state is a relation indexed by time [15].

3.4 Propositions

Proposition 1 (the Form captures every sequential algorithm). For every sequential algorithm A there is a Form instance P whose runs are the runs of A.

Proof sketch. Theorem 6.13 gives an ASM B with the states, initial states and one-step map of A. Set $E = \{\varepsilon\}$, $\delta(s, \varepsilon) = \tau_B(s)$, $D = (s_0, \varepsilon^n)$ and λ the identity. The fold after n ticks is $\tau_A^n(s_0)$. By Gurevich's Corollary 3.3, P and A have the same runs. ∎

Proposition 1b (interactive programs). Extend Σ with a nullary input symbol set to e before each step; then $\delta(s, e) = \tau(s[in := e])$ is a sequential algorithm over the extended vocabulary, and Proposition 1 applies. The construction is ours.

Proposition 1c (computability). Under the Dershowitz and Gurevich axiom, every Form instance computes a Turing-computable function [4, 16, 17]. The Form adds no power. It gives every program in the class one shape.

Proposition 1d (proofs are fits). "For each proof of a given proposition, there is a program of the corresponding type" [18]. A type is a specification and a proof is a fit that meets it. The correctness gate is the existence of that proof or of a passing acceptance test.

Proposition 2 (one generator). One function $G : \mathrm{Fit} \to \mathrm{Program}$ generates a runnable program for every fit and reads no domain. *Basis:* every program is a Form instance, so G emits a typed store, an event type, compiled rules, a log with a fold and an output dispatcher. Scaffolders such as Rails generators, JHipster and Yeoman are G restricted to small fit languages. Product-line engineering starts from "the systematic discovery and exploitation of commonality across related software systems" [8]; the commonality is the Form.

Proposition 3 (Mixture of Limits as extraction). Extraction with cost_J returns the least-joule term the extractor finds in the e-class of F. *Proof sketch.* The rewrites are sound, so every term in the e-class has F's observable behavior and passes the same gate. Extraction minimizes the cost, and the cost is joules. ∎ This is Mixture of Limits applied to code: keep the minimal fit that does the task, and the rest is waste.

Proposition 4 (replay determinism). If δ reads no wall clock, orders updates without names and logs every input, two replays of one D give identical state hashes at every step.

Proposition 5 (statechart projection is sound). Project each state of a run onto Σ_c. The result is a run of the emitted statechart under the same events.

Proposition 6 (coordination budget). Rules tagged monotone run on replicas without coordination; only non-monotone rules need a coordination point. *Basis:* the CALM theorem, "the programs that have consistent, coordination-free distributed implementations are exactly the programs that can be expressed in monotonic logic" [19].

Proposition 7 (intelligence as a joule derivative). Let X count tasks through the gate and J the joules spent. $\iota = dX/dJ$ is read in hindsight from D, which records every event and its `est_j`. Extraction under cost_J holds the gate fixed and does not raise J, so at equal X it does not lower ι (Agency, Section 2.3).

3.5 Graph, e-graph and statechart

Structure	Part of the Form	What it decides
Statechart	Guards of δ over Σ_c	Which transition fires
Dataflow graph	Read and write dependencies of δ and λ (Kahn process networks [20])	What runs, and in what order
E-graph	The equivalence class of the fit	Which equivalent fit runs

The statechart and the graph describe one fit. The e-graph describes every fit equivalent to it. Extraction chooses among them by joules.

---

4. The engine: gears, gate and receipts

4.1 The engine is a Form

RCE runs software as a graph of small deterministic handlers. A handler that cannot decide raises an escalation. A resolver answers the escalation once, and the answer is installed as a permanent handler, so the next case of the same shape runs with no resolver. The engine meets every Form invariant, each checked by a named test in `tests/form_laws.rs`:

Invariant	What the engine does
Reload is a verified fold	Reload reads the log verbatim, recomputes every hash and link, and folds the rest with δ. Nothing is re-stamped. A broken chain refuses to load.
Name-free tick order	Handlers run in topological rank order, then by a name-free node key. Outputs are staged and delivered after all handlers run, as in ASM semantics. Renaming every node gives the same anonymous state hash at every entry.
No silent drop	A full mailbox holds the message in an ordered queue and logs `Backpressure`. With capacity 1 and 10 tasks, 9 `Backpressure` entries are logged and all 10 tasks close.
Payloads are in the log	Inputs, ticks, external handler results, installs and faults are log entries. The state hash after every entry matches a fold from an empty engine, on 1,000 random logs.
Logical clock	Liveness counts ticks, not wall time.
Stable hashes	SHA-256 (Secure Hash Algorithm, 256-bit) over canonical JSON, keys sorted at every depth, for entries, states, shapes and rule ids.

4.2 The gears

Each gear is tried only when the gear below it does not pass the gate. That order is one fixed cascade, and the fixed cascade is one Mixture of Limits instance.

Gear	Method	Lineage
G0 Lookup	A shared handler library keyed by SHA-256 shape hash; for code repair, localization to ranked functions	Code search and retrieval [21]
G1 Rules	Deterministic, guarded repair templates and schema-derived validators, each checked for mutation sensitivity	PAR, TBar, AVATAR, FixMiner [22, 23, 24, 25]; codemods such as Coccinelle [26]
G2 Synthesis	Enumeration over the handler grammar, solvers and e-graphs	Sketching, SyGuS, egg [27, 9, 12]
G3 to G5 Models	A typed decision model, a local small model, then a remote model, last	Model cascades [28]

4.3 The gate

A candidate is promoted only when it passes the gate. The gate replays the node's recorded inputs, runs the candidate in shadow on fresh samples with a coverage bound (the rule of three at a declared error budget), checks the output schema, and requires the proof certificate of Section 5. A rejected candidate is logged with its counterexample.

4.4 Receipts

Every promotion, refusal, deferral and install is a mutation in an append-only log chained with SHA-256, so tampering with any past entry breaks the chain [29, 30]. Each handler's cost is priced from interpreter operation counts with the host cost table of FGIR, the native-code backend of flowG, and labeled `est_j`. Each foreman call carries its receipt. The ladder runs add a package energy counter on a subsample, labeled `reported_j`.

4.5 The Form compiler

`rce form` takes a specification and its fit and emits five targets from one Form IR:

```
.rce spec + fit
  lower ─► normalize ─► egg e-graph, equality saturation (+ admitted proposals)
  ─► extract (cost = est_j) ─► Form IR (typed relations, regions, rules with SHA-256 ids)
     ├─ Rust crate (pure step, typed state, SHA-256 log and fold, tests)
     │  └─ WASM (the same crate on wasm32-wasip1)
     ├─ Python (standard library only)
     ├─ XState v5 machine
     └─ SCXML document, executed by SCION
```

The IR fixes one behavior at every numeric edge: checked 64-bit integer overflow is an error and never wraps; integer `/` truncates toward zero ($-7/2 = -3$); `%` takes the dividend's sign ($-3 \bmod 2 = -1$); division by zero is an error; floats are IEEE 754 binary64 and a non-finite result is an

error; null and absent are distinct; time is the logical clock only. Every target is generated. None is authored.

A process, HTTP or closure handler has no term, so its logged results compile to a recorded oracle keyed by the SHA-256 of the canonical input. `rce scan DIR --form` lifts existing Rust, Python, TypeScript and Go into the Form's schema, and every lifted element is labeled as a syntactic lift.

---

5. Verification

5.1 Differential testing across five targets

`tests/form_targets.rs` drives an eleven-operator arithmetic specification with 29 edge events plus 500 seeded random edge values. The generated Rust, Python, WASM, XState and SCXML targets print the reference trace byte for byte, including the arithmetic edges. This is differential testing in the line of McKeeman and of Csmith [86]: agreement among independent implementations on a stated input set. It is not translation validation [85] and not a verified compiler. It covers the eleven operators and the 529 events in the test, and it says nothing about inputs outside them. The Z3 and Kani results below are the verification.

5.2 Rewrites proved in Z3

Verified rewrite rules have a lineage. Ruler infers and checks rewrite rules by equality saturation with no model in the loop [87]. The 3 of 16 model-proposed rules admitted in Section 5.5 are therefore a statement about the gate, which refused 13, and not a claim that a model is the best rule source.

Every built-in e-graph rewrite is proved in the Z3 SMT solver (satisfiability modulo theories) [31] against a quantifier-free encoding of the Form's term evaluation and arithmetic: values as a datatype, 64-bit bit-vectors with checked overflow, signed division and remainder with the dividend's sign, and binary64 floats. A rewrite that changes overflow, escalation order or a message gets no proof. A proposed equivalence enters the e-graph only with a proof.

5.3 Kani proofs of the kernel

The engine's kernel (hash chain, fold, name-free positions and handler evaluation) is checked with the Kani model checker for Rust [32]. Handler evaluation is the fixed point of a non-recursive step, and Kani proves that step total and deterministic per constructor, by structural induction. Ten harnesses are verified, with 0 of 505 checks failed:

Harness	Property
`chain_appended_verifies`	An appended hash chain verifies
`chain_detects_tampering`	A changed entry breaks the chain
`fold_equals_replay`	The fold equals replay

Harness	Property
`fold_rejects_wrong_state`	A fold that disagrees with the stored state is refused
`position_is_name_independent`	Rule positions do not depend on node names
`reason_tokens_are_name_independent`	Reasons are name-free by whole tokens
`handler_core_leaves`	Leaf handlers are total and deterministic
`handler_core_validate`	Validation is total and deterministic
`handler_core_pipeline`	Pipelines are total and deterministic
`handler_core_ifhas`	Guards are total and deterministic

5.4 Promotion gate on the proof certificate

The Kani run writes a certificate whose hash is stored in `proofs/kani.cert`. The engine, the log, rule ids and handler evaluation route through the proved kernel, and promotion requires the certificate.

5.5 The model as a proposer behind the check

A model may propose equivalences. It may not admit them. With a local 1.5-billion-parameter coding model (Qwen2.5-Coder-1.5B under llama.cpp) on the demonstration specification, the proposer offered 16 equivalences. The check admitted 3, by randomized trials and a Z3 proof, and rejected 13. A deterministic proposer of 87 familiar identities had 9 admitted, all structural, and every arithmetic identity rejected for fault order, non-numbers or overflow.

5.6 Brownfield agreement

The lift reads the same order machine from Rust, Python, TypeScript and Go fixtures, runs the original code on 40 seeded sequences of 30 calls, and agrees on all 1,200 calls per language. A frozen benchmark, preregistered before any result, re-expressed three existing sales-outreach programs as Forms. The Forms agree with the original code on 4,500 of 4,500 recorded inputs, and extraction lowers `est_j` on each (for example 1.69×10^{-8} J to 1.49×10^{-8} J).

---

6. Results

Method. Each corpus is shuffled with a fixed seed. The first 2,000 positions of each shuffle, and for L0 three whole repositories (commons-lang, requests, flask), form the development split, on which the gears may be tuned; development results are not reported. Every remaining task is in the test split. Before a test run the engine is committed and its binary hash recorded, and no gear changes during the run. Every row in this section is a frozen-engine result on test tasks only, with 0 model calls. The preregistration (`PREREG.json`), the per-task rows and the report script are in the public repository.

Gate. At L0 and L1 the location is given and the gate is token identity: the repaired line or statement must equal the developer's, token for token. This is stricter than passing the tests, because a patch can pass a weak test suite and still be wrong. Qi and colleagues found that most patches GenProg reported as fixes were plausible but incorrect [92]; overfitting to tests is documented in search-based and semantics-based repair [93, 94] and addressed with generated tests [95]; tools score higher on the benchmark they were tuned on [96]; and fault localization biases comparisons [97]. Token identity removes the plausibility gap on these rungs, and giving the location removes the localization term, so the rate measures the repair gears alone. L2 and above use the tests as the gate and report plausible and correct counts separately (Section 8.1).

6.1 L0: one-token faults

The L0 corpus is every PIT-style mutant [109] with a valid replacement token in nine real repositories at pinned commits, five Java and four Python: 520,821 test tasks out of 547,448. The engine (commit `65571d0`, gears G0-T and G1-J) restores 520,541, a rate of 0.9995 (95% Wilson interval 0.9994 to 0.9995) at a median of 2.3 search bits. Of the 280 unsolved, 108 are mutated digits inside floating-point literals and 2 are inside Python set literals. A float-literal gear (G0-F, commit `ecfe5a1`) is tested on four repositories never used for tuning (commons-text, commons-io, more-itertools, httpx): 23,015 of 23,015 tasks, 1.0000 (0.9998 to 1.0000), a median of 2.0 search bits. Mutants are a stand-in for real one-token faults that Just and colleagues found to be coupled with real faults [110].

6.2 L1: one-statement bugs from real histories

L1 uses two public datasets of real single-statement bug fixes mined from open-source histories. ManySStuBs4J 2.0 [107] gives 24,072 Java test tasks; the engine (commit `f2701b7`) resolves 9,288, a rate of 0.386 (0.380 to 0.392). TSSB-3M v1.0.0 [108] gives 157,089 Python test tasks from shards 14 and 15; the same engine resolves 27,266, a rate of 0.174 (0.172 to 0.175). The engine receives the 7-line hunk only, so the Java rate is a hunk-only result; the full-file result is the next row of this table. Per-category rates are in Section 2.3 and the full per-stratum tables are in the repository. These datasets have no published repair leaderboard under this gate (hunk only, token identity, location given), so no learned-repair number is set beside them; Defects4J, which does have one, is compared with TBar in Section 8.1.

6.3 Energy

Every row carries `est_j`, an estimate of CPU seconds times 10 W: 0.40 J per solved task at L0, 0.13 on the fresh repositories, 0.10 on ManySStuBs4J and 0.27 on TSSB-3M. A subsample was read on RAPL [105], the processor's on-chip package counter, on a Linux laptop with an Intel i9-13900H (hostname `jetson-hub`; not an NVIDIA Jetson). The harness ran on 4 pinned CPUs, and `reported_j` is the package delta over the whole run, Python driver included, less idle power from a 10 s sample taken just before, times the run's duration. Continuous-integration runners were up, so idle was 40 W to 44 W and the uncertainty is about ±1 W times the duration, ±10 J on these totals.

Subsample (test split)	Tasks	Solved	Raw J	reported_j	reported_j per solved task	Seconds
L0, gson mutants (seeded, first 3,000)	3,000	3,000	648.3	167.5	0.056	11.8
L1, ManySStuBs4J positions 2,000 to 3,999	2,000	690	377.2	17.9	0.026	8.2

The L1 subsample builds its project index from its 2,000 tasks only, so its solve rate (34.5%) is below the full run's. `measured_j`, a wall meter on a named rail, is empty in every row; its protocol is Section 8.3.

6.4 The soaks converge

The soak runs the five example specifications under two resolvers for 20 rounds each and reports `est_j` per closed task. In the 2026-10-09 run, every specification reaches a round with zero new escalations, in round 4 to 7 for both resolvers, and from that round the all-in `est_j` per closed task equals the handler floor: about 2 to 5 nanojoules per task. The gear stack resolved every case from G0 and G1, with zero model calls. In the 2026-10-10 live2 run, on live traffic only, all 12 runs reach a quiet round before round 20 (web-api 4, joule-db-core 7, follow-bible-ingest 6, follow-bible-ingest-schemas 8, ask-server 5, soak-g2-schemas 5), and every run closes tasks through the gate. `measured_j` is empty in every row.

6.5 Model calls in the first soak

The first end-to-end soak ran `rce auto examples/web-api.rce --foreman federated --rounds 5` on 2026-04-25, with a federated model resolver over 12 providers and auto-approval. Each round pumps 5 traffic events into the source nodes, ticks, resolves every escalation and approves the queued mutations. Round 1 had 10 new escalations, 10 promotions and 10 model calls. Rounds 2 to 5 had 0 of each, and the log integrity check passed. That is 10 model calls in 5 rounds, against 50 if the model were called every round. The counts are as printed in `SOAK_RESULTS.md` in the crate. No energy was read in this run.

The same file reports a re-run over three specifications. Two converged: web-api in 10 model calls and joule-db-core in 5, over 5 rounds. The third, follow-bible-ingest, held at 5 escalations a round from one stub dependency, 115 model calls over 20 rounds. With stuck-input deferral, a repeated input shape is deferred to a human queue without a model call, and the same 20 rounds took 20 model calls.

---

7. Reuse evidence

The second path rests on an engineering fact: code is reused at every scale. Each method below measures it differently.

Method	Source	Finding
File-level clone mapping	DéjàVu, Lopes et al. 2017 [33]	Of 428 million files in 4.5 million non-fork GitHub projects, 85 million are unique: "70% of the code on GitHub consists of clones of previously created files." In JavaScript, 6% of files are distinct.
Line-level redundancy	Gabel and Su 2010 [34]	Against 420 million lines, "between 50% and 100% of each project is redundant" at about one line.
Statistical language models	Hindle et al. 2012 [35]	Code is "very repetitive, and in fact even more so than natural languages."
Corpus near-deduplication	The Stack [36]	Near-deduplication removes 54.7% of The Stack v1 (6.4 TB to 2.9 TB) and 52.4% of v2 (67.5 TB to 32.1 TB).
Whole-archive preservation	Software Heritage	About 30 billion source files from 445 million origins, each stored once by the hash of its bytes.
Mined fix patterns	PAR, TBar [22, 23]	PAR found "several common fix patterns" in more than 60,000 human patches. TBar's 15 pattern categories fix 43 Defects4J bugs correctly in the standard pipeline, and 74 of 101 with perfect fault localization, with no learning.
Cross-language tasks	Rosetta Code	1,360 tasks, each one concept implemented across many of 1,009 known languages.

Each row is an input to the engine. Clones and dedup say a fit usually exists, which is G0. Recurring fixes say adaptation is mostly a small catalog of guarded rewrites, which is G1. Line-level redundancy shrinks the search space of G2. The L0 and L1 rungs measure this at scale on held-out splits (Section 6). Two cautions sharpen the reading. Duplicates inflate retrieval and model scores unless removed [104]. And buggy lines are less natural than fixed ones [45], so the bug itself is the case lookup is least able to predict; the fix shape, not the buggy line, is what G0 and G1 must key on. The rule-mining line that does this at scale, from GenProg's generate-and-validate search [88] and Prophet's learned correctness model [89] to Getafix at Facebook [90] and Sorald's model-free repair of static-analysis violations [91], derives templates from disjoint history and reports precision on unseen cases [98]. That is the standard Section 8.4 adopts.

---

8. Measurement protocols for what comes next

Each protocol is fixed before its run: preregistration committed, seeds and solver commit recorded, every artifact kept. Gate first, joules second, labels always.

8.1 Defects4J

Defects4J holds 854 active real Java bugs, each with reproducing tests [37].

1. Develop on a 100-bug subset, offline, through `defects4j checkout`; report held-out results only, on a disjoint set of 288 bugs frozen before the run (`FROZEN-heldout-v1`).
2. Run G0, then G1, then G2, then a local model last. Record the gear that closed each bug.

3. Gate: the triggering tests and the full suite pass. Report plausible and correct counts separately; correct means equivalent to the developer patch on review.
4. Report model calls per correct fix and joules per correct fix, labeled.
5. Reference points: TBar's 43 correct fixes and its 74 of 101 under perfect localization.
6. Contamination check: flag every bug whose fix appears verbatim in G0's index, and report with and without flagged bugs.

8.2 SWE-bench Lite, offline

SWE-bench Lite is a 300-task subset of real GitHub issues resolved by hidden tests [38].

1. Use the official Docker harness with no network.
2. First run: G0 and G1 only, zero model calls. This is the floor of solving without a model.
3. Second run: add a local model as the last gear.
4. Report the resolve rate, model calls per resolved task and joules per resolved task, labeled.
5. Contamination check as in 8.1. SWE-Bench+ found that 32.67% of successful patches had the solution in the issue text [39].

8.3 A wall meter

1. Attach a wall meter or a shunt meter on the host's supply, logged at a stated rate with its clock synchronized to the run log.
2. Idle machine: CI runners and other builds stopped, and a quiet window gate checked before each block and recorded.
3. Subtract a measured idle baseline, and state the method.
4. Fill `measured_j` from the meter alone. Report `reported_j` (RAPL) and `est_j` beside it in separate columns. Never sum across labels.
5. Repeat the RAPL subsamples of Section 6.3 under the meter, then the Defects4J held-out set.

8.4 A held-out run with the gears frozen

This is the method of Section 6, extended to new corpora.

1. Freeze G0, G1 and G2 at a tagged commit, record the binary hash, and publish the rule set with its authoring timestamps.
2. Draw a disjoint held-out slice: the Rust and Python instances of Multi-SWE-bench [102] and BugsInPy [103], de-duplicated against G0's index [104].
3. Preregister the slice, the seeds and the hypotheses before the first run.
4. Report resolved, plausible and byte-identical counts separately, each with a 95% interval.

8.5 Baselines in the same table

1. Model-free: `cargo fix` and clippy [106], and Sorald or a TBar port [91].
2. Cheap model: Agentless with a local model [99].

3. Remote model: an agent with a frontier model, its energy estimated from provider disclosures and labeled `est_j`.
4. Report cost-of-pass [101] in joules and in dollars for every row, with cost controlled as Kapoor and colleagues require [100].

Proposition 8 (floor approach). As G0 and G1 coverage grows, joules per solved task approach the gold floor of applying a known fix and running the tests. The ladder rungs and the Defects4J held-out set under the wall meter test it.

---

9. Ties to the companion papers

Mixture of Limits. Metered Commit Gates, Part II states the philosophy: a minimal amount of anything is needed to accomplish a task, and any more is waste. RCE applies it three times. The gear order runs the cheapest sufficient gear first. Promotion pays a resolver once per new case, after which every repeat call would be waste. Extraction keeps the least-joule fit among equivalent fits (Proposition 3).

Metered Commit Gates. Part I says a proposal may not authorize itself. In RCE a model proposes and the gate commits: a candidate enters only with tests passed, a coverage bound met and a proof certificate, and every decision is a receipt in the hash chain.

The Universal Law of Agency. The agency paper defines intelligence as the derivative of agency in joules, $\iota = dX/dJ$, read in hindsight. The RCE log is that hindsight record: X is tasks through the gate and J is their joules, labeled. A deferral with a receipt is the refusal clause of the gate.

Task Satiation. Task Satiation vs. Aggregate Rebound in AI Demand says that once a task's completion predicate is true, more synthesis on that task buys nothing. RCE is per-task satiation inside one system: once a handler covers a case, further inference on that case is zero. The aggregate demand question belongs to that paper and to the economics companion.

---

10. Falsifiers

Fixed before the held-out run of Section 8.4.

ID	Observation	Defeats
RCE-F1	On the held-out slice, G0 to G2 with frozen rules resolve under 10% of tasks with 0 model calls	The claim that most fits already exist for repair at this scale
RCE-F2	Byte-identical or equivalent patches are under half of the resolved held-out tasks	The reading of the gate as more than plausibility on the covered tasks
RCE-F3	Under the wall meter, gross joules per solved task of the RCE solver exceed those of the cheapest model baseline on the	The second path's joule advantage on covered tasks

ID	Observation	Defeats
	tasks both solve	
RCE-F4	In a soak with a priced model resolver, model calls per round do not reach 0 within 20 rounds on any specification without a stuck input	Compiling the model out of the loop

11. Work to finish

Each item is open work with its protocol named.

1. Held-out results on L2 to L4, the Defects4J set of 288 bugs (Section 8.1), and on L5 to L7 (Section 8.2).
2. Baselines in one table, cost-of-pass in joules and dollars (Section 8.5).
3. The first `measured_j`: a wall meter on an idle host, at least 10 repeats, gross and net side by side (Section 8.3).
4. The full-file L1 Java result, with a type-indexed API index as the next gear (Section 6.2).
5. The 626-language survey published with its method (Figure 1).
6. New held-out corpora with the gears frozen: Multi-SWE-bench and BugsInPy (Section 8.4).

12. Conclusion

A program is a database functional state machine, and RCE treats coding as a conversion from high-entropy intent to that low-entropy Form. Correctness is the gate and joules are the ranking. On held-out test splits with a frozen engine and no model call, the gears repair 99.95% of 520,821 one-token faults and all 23,015 in repositories never tuned on, 38.6% of 24,072 single-statement Java bugs and 17.4% of 157,089 Python bugs, at hundredths of a reported joule per solved task on a package counter; in the first soak the model was called 10 times in round 1 and not again. The verification layer proves the rewrites in Z3 and the kernel in Kani, and differential testing ties five targets to one trace. What remains is the upper rungs, baselines in the same table, and a wall meter. Those protocols are fixed in Sections 8.3 to 8.5 and the falsifiers in Section 10.

References

[1] C. E. Shannon, “Prediction and Entropy of Printed English,” *Bell System Technical Journal* 30(1), 1951. doi:10.1002/j.1538-7305.1951.tb01366.x

[2] Y. Gurevich, “Sequential Abstract State Machines Capture Sequential Algorithms,” *ACM Transactions on Computational Logic* 1(1), 2000. doi:10.1145/343369.343384

[3] A. Blass and Y. Gurevich, “Abstract State Machines Capture Parallel Algorithms,” *ACM Transactions on Computational Logic*, 2003. doi:10.1145/937555.937561

[4] N. Dershowitz and Y. Gurevich, “A Natural Axiomatization of Computability and Proof of Church’s Thesis,” *Bulletin of Symbolic Logic*, 2008. doi:10.2178/bsl/1231081370

[5] E. F. Codd, “A Relational Model of Data for Large Shared Data Banks,” *Communications of the ACM*, 1970. doi:10.1145/362384.362685

[6] G. H. Mealy, “A Method for Synthesizing Sequential Circuits,” *Bell System Technical Journal*, 1955. doi:10.1002/j.1538-7305.1955.tb03788.x

[7] E. F. Moore, “Gedanken-Experiments on Sequential Machines,” *Automata Studies*, 1956. doi:10.1515/9781400882618-006

[8] K. Kang et al., “Feature-Oriented Domain Analysis (FODA) Feasibility Study,” CMU/SEI-90-TR-021, 1990.

[9] R. Alur et al., “Syntax-Guided Synthesis,” FMCAD 2013. doi:10.1109/FMCAD.2013.6679385

[10] D. Harel, “Statecharts: A Visual Formalism for Complex Systems,” *Science of Computer Programming*, 1987. doi:10.1016/0167-6423(87)90035-9

[11] R. Tate, M. Stepp, Z. Tatlock and S. Lerner, “Equality Saturation,” POPL 2009. doi:10.1145/1480881.1480915

[12] M. Willsey et al., “egg: Fast and Extensible Equality Saturation,” POPL 2021. doi:10.1145/3434304

[13] B. Moseley and P. Marks, “Out of the Tar Pit,” 2006.

[14] M. Fowler, “Event Sourcing,” 2005. https://martinfowler.com/eaaDev/EventSourcing.html

[15] P. Alvaro et al., “Dedalus: Datalog in Time and Space,” LNCS 2011. doi:10.1007/978-3-642-24206-9_16

[16] A. M. Turing, “On Computable Numbers,” *Proceedings of the London Mathematical Society*, 1937. doi:10.1112/plms/s2-42.1.230

[17] A. Church, “An Unsolvable Problem of Elementary Number Theory,” *American Journal of Mathematics*, 1936. doi:10.2307/2371045

[18] P. Wadler, “Propositions as Types,” *Communications of the ACM*, 2015. doi:10.1145/2699407

[19] J. M. Hellerstein and P. Alvaro, “Keeping CALM,” *Communications of the ACM*, 2020. doi:10.1145/3369736

[20] G. Kahn, “The Semantics of a Simple Language for Parallel Programming,” IFIP 1974, pp. 471 to 475.

[21] H. Husain et al., “CodeSearchNet Challenge,” 2019. arXiv:1909.09436

[22] D. Kim, J. Nam, J. Song and S. Kim, “Automatic Patch Generation Learned from Human-Written Patches” (PAR), ICSE 2013. doi:10.1109/ICSE.2013.6606626

[23] K. Liu et al., “TBar: Revisiting Template-based Automated Program Repair,” ISSTA 2019. arXiv:1903.08409

[24] K. Liu et al., “AVATAR: Fixing Semantic Bugs with Fix Patterns of Static Analysis Violations,” SANER 2019. arXiv:1812.07270

[25] A. Koyuncu et al., “FixMiner: Mining Relevant Fix Patterns for Automated Program Repair,” 2018. arXiv:1810.01791

[26] Y. Padioleau et al., “Documenting and Automating Collateral Evolutions in Linux Device Drivers” (Coccinelle), EuroSys 2008. doi:10.1145/1352592.1352618

[27] A. Solar-Lezama et al., “Combinatorial Sketching for Finite Programs,” ASPLOS 2006. doi:10.1145/1168857.1168907

[28] L. Chen, M. Zaharia and J. Zou, “FrugalGPT,” 2023. arXiv:2305.05176

[29] S. Haber and W. S. Stornetta, “How to Time-Stamp a Digital Document,” *Journal of Cryptology*, 1991. doi:10.1007/BF00196791

[30] S. A. Crosby and D. S. Wallach, “Efficient Data Structures for Tamper-Evident Logging,” USENIX Security 2009.

[31] L. de Moura and N. Bjørner, “Z3: An Efficient SMT Solver,” TACAS 2008. doi:10.1007/978-3-540-78800-3_24

[32] Kani Rust Verifier. https://github.com/model-checking/kani

[33] C. V. Lopes et al., “DéjàVu: A Map of Code Duplicates on GitHub,” OOPSLA 2017. doi:10.1145/3133908

[34] M. Gabel and Z. Su, “A Study of the Uniqueness of Source Code,” FSE 2010. doi:10.1145/1882291.1882315

[35] A. Hindle et al., “On the Naturalness of Software,” ICSE 2012. doi:10.1109/ICSE.2012.6227135

[36] D. Kocetkov et al., “The Stack: 3 TB of Permissively Licensed Source Code,” 2022. arXiv:2211.15533; The Stack v2 dataset card, https://huggingface.co/datasets/bigcode/the-stack-v2

[37] R. Just, D. Jalali and M. D. Ernst, “Defects4J: A Database of Existing Faults to Enable Controlled Testing Studies for Java Programs,” ISSTA 2014. doi:10.1145/2610384.2628055

[38] C. E. Jimenez et al., “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?” 2023. arXiv:2310.06770

[39] R. Aleithan et al., “SWE-Bench+: Enhanced Coding Benchmark for LLMs,” 2024. arXiv:2410.06992

[40] C. E. Shannon, “A Mathematical Theory of Communication,” *Bell System Technical Journal* 27, 1948. doi:10.1002/j.1538-7305.1948.tb01338.x

[41] A. N. Kolmogorov, “Three approaches to the quantitative definition of information,” *Problemy Peredachi Informatsii* 1(1), 1965 (in Russian); English translation, *International Journal of Computer Mathematics* 2, 1968. doi:10.1080/00207166808803030

[42] R. J. Solomonoff, “A formal theory of inductive inference. Part I,” *Information and Control* 7(1), 1964. doi:10.1016/S0019-9958(64)90223-2

[43] G. J. Chaitin, “A Theory of Program Size Formally Identical to Information Theory,” *Journal of the ACM* 22(3), 1975. doi:10.1145/321892.321894

[44] J. Rissanen, “Modeling by shortest data description,” *Automatica* 14(5), 1978. doi:10.1016/0005-1098(78)90005-5

[45] B. Ray, V. Hellendoorn, S. Godhane, Z. Tu, A. Bacchelli and P. Devanbu, “On the ‘naturalness’ of buggy code,” ICSE 2016. doi:10.1145/2884781.2884848

[46] Z. Tu, Z. Su and P. Devanbu, “On the localness of software,” FSE 2014. doi:10.1145/2635868.2635875

[47] M. Allamanis, E. T. Barr, P. Devanbu and C. Sutton, “A Survey of Machine Learning for Big Code and Naturalness,” *ACM Computing Surveys* 51(4), 2018. doi:10.1145/3212695

[48] 陈浙哲, 刘忠鑫, 鄢萌, 徐洲, 夏鑫, 雷晏 (Z. Chen, Z. Liu, M. Yan, Z. Xu, X. Xia and Y. Lei), “代码自然性及其应用研究进展” (Research progress of code naturalness and its application), *软件学报 (Journal of Software)* 33(8), 3015 to 3034, 2022. doi:10.13328/j.cnki.jos.006355

[49] G. Sridhara, V. S. Sinha and S. Mani, “Naturalness of Natural Language Artifacts in Software,” ISEC 2015 (India Software Engineering Conference). doi:10.1145/2723742.2723758

[50] P. Devanbu, “On the naturalness of software,” keynote, ISEC 2013 (India Software Engineering Conference). doi:10.1145/2442754.2442763

[51] A. Z. H. Yang, S. Kolak, V. J. Hellendoorn, R. Martins et al., “Revisiting Unnaturalness for Automated Program Repair in the Era of Large Language Models,” ICSE 2025. doi:10.1109/ICSE55347.2025.00089

[52] E. W. Dijkstra, “Guarded commands, nondeterminacy and formal derivation of programs,” *Communications of the ACM* 18(8), 1975. doi:10.1145/360933.360975

[53] N. Wirth, “Program development by stepwise refinement,” *Communications of the ACM* 14(4), 1971. doi:10.1145/362575.362577

[54] C. A. R. Hoare, “An axiomatic basis for computer programming,” *Communications of the ACM* 12(10), 1969. doi:10.1145/363235.363259

[55] J.-R. Abrial, *The B-Book: Assigning Programs to Meanings*, Cambridge University Press, 1996. doi:10.1017/CBO9780511624162

[56] J.-R. Abrial, *Modeling in Event-B: System and Software Engineering*, Cambridge University Press, 2010. doi:10.1017/CBO9781139195881

[57] R.-J. Back and J. von Wright, *Refinement Calculus: A Systematic Introduction*, Springer, 1998. doi:10.1007/978-1-4612-1674-2

[58] F. L. Bauer and H. Wössner, *Algorithmic Language and Program Development*, Springer, 1982. doi:10.1007/978-3-642-61807-9

[59] S. Gulwani, “Automating string processing in spreadsheets using input-output examples,” POPL 2011. doi:10.1145/1926385.1926423

[60] S. Gulwani, O. Polozov and R. Singh, “Program Synthesis,” *Foundations and Trends in Programming Languages* 4(1-2), 2017. doi:10.1561/2500000010

[61] F. P. Brooks, “No Silver Bullet: Essence and Accidents of Software Engineering,” *Computer* 20(4), 1987. doi:10.1109/MC.1987.1663532

[62] P. Naur, “Programming as theory building,” *Microprocessing and Microprogramming* 15(5), 1985. doi:10.1016/0165-6074(85)90032-8

[63] M. M. Lehman, “Programs, life cycles, and laws of software evolution,” *Proceedings of the IEEE* 68(9), 1980. doi:10.1109/PROC.1980.11805

[64] P. Zave and M. Jackson, “Four dark corners of requirements engineering,” *ACM Transactions on Software Engineering and Methodology* 6(1), 1997. doi:10.1145/237432.237434

[65] R. Landauer, “Irreversibility and Heat Generation in the Computing Process,” *IBM Journal of Research and Development* 5(3), 1961. doi:10.1147/rd.53.0183

[66] C. H. Bennett, “Logical Reversibility of Computation,” *IBM Journal of Research and Development* 17(6), 1973. doi:10.1147/rd.176.0525

[67] M. Franz and T. Kistler, “Slim binaries,” *Communications of the ACM* 40(12), 1997. doi:10.1145/265563.265576

[68] C. W. Fraser, “Automatic inference of models for statistical code compression,” PLDI 1999. doi:10.1145/301618.301672

[69] P. Cousot and R. Cousot, “Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints,” POPL 1977. doi:10.1145/512950.512973

[70] Y. Futamura, “Partial evaluation of computation process: an approach to a compiler-compiler,” *Systems, Computers, Controls* 2(5), 1971; reprinted in *Higher-Order and Symbolic Computation* 12(4), 1999. doi:10.1023/A:1010095604496

[71] A. P. Ershov, “Mixed computation: potential applications and problems for study,” *Theoretical Computer Science* 18(1), 1982. doi:10.1016/0304-3975(82)90111-6

[72] W. A. Howard, “The formulae-as-types notion of construction,” in J. P. Seldin and J. R. Hindley (eds.), *To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*, Academic Press, 1980.

[73] 土居真之, 肥後芳樹, 有馬諒, 下仲健斗, 楠本真二 (M. Doi, Y. Higo, R. Arima, K. Shimonaka and S. Kusumoto), “ソースコードの「自然さ」を利用した自動生成ファイルの特定” (Identifying auto-generated files using the naturalness of source code), 電子情報通信学会技術研究報告 (IEICE Technical Report) 117(380), 19 to 24, January 2018. https://sdl.ist.osaka-u.ac.jp/pman/pman3.cgi?D=517

[74] M. Doi, Y. Higo, R. Arima, K. Shimonaka and S. Kusumoto, “On the naturalness of auto-generated code: can we identify auto-generated code automatically?” ICPC 2018. doi:10.1145/3196321.3196356

[75] F. V. Arrebola, “Modelo de sugestão de código-fonte baseado em modelos de linguagem organizados em um contexto hierárquico” (A source code suggestion model based on language models organized in a hierarchical context), doctoral thesis in electrical engineering, Fundação Educacional Inaciana Padre Sabóia de Medeiros (FEI), Brazil, 2018. Advisor R. A. C. Bianchi.

[76] G. Febres, K. Jaffé and C. Gershenson, “Complexity measurement of natural and artificial languages,” *Complexity* 20(6), 2015. doi:10.1002/cplx.21529

[77] M. Rahman, D. Palani and P. C. Rigby, “Natural Software Revisited,” ICSE 2019. doi:10.1109/ICSE.2019.00022

[78] A. Torres, S. Baltes, C. Treude and M. Wagner, “Applying Information Theory to Software Evolution,” NLBSE 2023. doi:10.1109/NLBSE59153.2023.00017

[79] A. Blass, N. Dershowitz and Y. Gurevich, “When are two algorithms the same?,” *Bulletin of Symbolic Logic* 15(2), 2009. arXiv:0811.0811. https://arxiv.org/abs/0811.0811

[80] Y. N. Moschovakis, “What is an algorithm?,” in *Mathematics Unlimited: 2001 and Beyond*, Springer, 2001. https://www.math.ucla.edu/~ynm/papers/eng.pdf

[81] M. Y. Vardi, “What is an algorithm?,” *Communications of the ACM* 55(3), 2012. doi:10.1145/2093548.2093549

[82] S. Laddad, A. Cheung, J. M. Hellerstein and M. Milano, “Flo: a Semantic Foundation for Progressive Stream Processing,” POPL 2025. arXiv:2411.08274. https://arxiv.org/abs/2411.08274

[83] The Elm Architecture, Elm guide. https://guide.elm-lang.org/architecture/

[84] Temporal, “Workflows” (event history), documentation. https://docs.temporal.io/workflows

[85] A. Pnueli, M. Siegel and E. Singerman, “Translation validation,” TACAS 1998. doi:10.1007/BFb0054170

[86] X. Yang, Y. Chen, E. Eide and J. Regehr, “Finding and understanding bugs in C compilers,” PLDI 2011. doi:10.1145/1993498.1993532

[87] C. Nandi, M. Willsey, A. Zhu et al., “Rewrite Rule Inference Using Equality Saturation,” OOPSLA 2021. arXiv:2108.10436. https://arxiv.org/abs/2108.10436

[88] C. Le Goues, T. Nguyen, S. Forrest and W. Weimer, “GenProg: A Generic Method for Automatic Software Repair,” *IEEE TSE* 38(1), 2012. doi:10.1109/TSE.2011.104

[89] F. Long and M. Rinard, “Automatic patch generation by learning correct code,” POPL 2016. doi:10.1145/2837614.2837617

[90] J. Bader, A. Scott, M. Pradel and S. Chandra, “Getafix: Learning to Fix Bugs Automatically,” OOPSLA 2019. arXiv:1902.06111. https://arxiv.org/abs/1902.06111

[91] K. Etemadi, N. Harrand, S. Larsen et al., “Sorald: Automatic Patch Suggestions for SonarQube Static Analysis Violations,” *IEEE TDSC*, 2022. arXiv:2103.12033. https://arxiv.org/abs/2103.12033

[92] Z. Qi, F. Long, S. Achour and M. Rinard, “An analysis of patch plausibility and correctness for generate-and-validate patch generation systems,” ISSTA 2015. doi:10.1145/2771783.2771791

[93] E. K. Smith, E. T. Barr, C. Le Goues and Y. Brun, “Is the cure worse than the disease? Overfitting in automated program repair,” ESEC/FSE 2015. doi:10.1145/2786805.2786825

[94] X.-B. D. Le, F. Thung, D. Lo and C. Le Goues, “Overfitting in semantics-based automated program repair,” *Empirical Software Engineering* 23, 2018. doi:10.1007/s10664-017-9577-2

[95] Y. Xiong, X. Liu, M. Zeng, L. Zhang and G. Huang, “Identifying patch correctness in test-based program repair,” ICSE 2018. doi:10.1145/3180155.3180182

[96] T. Durieux, F. Madeiral, M. Martinez and R. Abreu, “Empirical Review of Java Program Repair Tools: A Large-Scale Experiment on 2,141 Bugs and 23,551 Repair Attempts,” ESEC/FSE 2019. arXiv:1905.11973. https://arxiv.org/abs/1905.11973

[97] K. Liu, A. Koyuncu, T. F. Bissyandé et al., “You Cannot Fix What You Cannot Find! An Investigation of Fault Localization Bias in Benchmarking Automated Program Repair Systems,” ICST 2019. arXiv:1812.07283. https://arxiv.org/abs/1812.07283

[98] M. Monperrus, “Automatic Software Repair: A Bibliography,” *ACM Computing Surveys* 51(1), 2018. doi:10.1145/3105906

[99] C. S. Xia, Y. Deng, S. Dunn and L. Zhang, “Agentless: Demystifying LLM-based Software Engineering Agents,” 2024. arXiv:2407.01489. https://arxiv.org/abs/2407.01489

[100] S. Kapoor, B. Stroebl, Z. S. Siegel, N. Nadgir and A. Narayanan, “AI Agents That Matter,” 2024. arXiv:2407.01502. https://arxiv.org/abs/2407.01502

[101] M. H. Erol, B. El, M. Suzgun, M. Yuksekgonul and J. Zou, “Cost-of-Pass: An Economic Framework for Evaluating Language Models,” 2025. arXiv:2504.13359. https://arxiv.org/abs/2504.13359

[102] D. Zan, Z. Huang, W. Liu et al., “Multi-SWE-bench: A Multilingual Benchmark for Issue Resolving,” 2025. arXiv:2504.02605. https://arxiv.org/abs/2504.02605

[103] R. Widyasari, S. Q. Sim, C. Lok et al., “BugsInPy: a database of existing bugs in Python programs to enable controlled testing and debugging studies,” ESEC/FSE 2020. doi:10.1145/3368089.3417943

[104] M. Allamanis, “The Adverse Effects of Code Duplication in Machine Learning Models of Code,” Onward! 2019. arXiv:1812.06469. https://arxiv.org/abs/1812.06469

[105] K. N. Khan, M. Hirki, T. Niemi, J. K. Nurminen and Z. Ou, “RAPL in Action: Experiences in Using RAPL for Power Measurements,” *ACM TOMPECS* 3(2), 2018. doi:10.1145/3177754

[106] The Rust project, rustfix and `cargo fix`. https://github.com/rust-lang/rustfix

[107] R.-M. Karampatsis and C. Sutton, “How Often Do Single-Statement Bugs Occur? The ManySStuBs4J Dataset,” MSR 2020. doi:10.1145/3379597.3387491

[108] C. Richter and H. Wehrheim, “TSSB-3M: Mining single statement bugs at massive scale,” MSR 2022. doi:10.1145/3524842.3528505

[109] H. Coles, T. Laurent, C. Henard, M. Papadakis and A. Ventresque, “PIT: a practical mutation testing tool for Java,” ISSTA 2016. doi:10.1145/2931037.2948707

[110] R. Just, D. Jalali, L. Inozemtseva, M. D. Ernst, R. Holmes and G. Fraser, “Are mutants a valid substitute for real faults in software testing?” FSE 2014. doi:10.1145/2635868.2635929